

COE 379L Project 02: Hurricane Harvey Building Damage Classification Using Convolutional Neural Networks

For this project, my goal was to build and deploy a neural network capable of automatically detecting whether a building in a satellite image had been damaged after Hurricane Harvey. The dataset provided by the course contained post-disaster aerial imagery from across Texas divided into two labeled categories—"damage" and "no_damage." My objective was to preprocess this data for machine learning, experiment with multiple neural-network architectures, evaluate them quantitatively, and then deploy the best-performing model in a functional inference server containerized through Docker. The workflow mirrored that of a real end-to-end ML engineer: preparing raw data, designing and validating models empirically, and ensuring that the final deployed system was reproducible, efficient, and API-compliant.

The dataset included 14,170 images of damaged buildings and 7,152 images of non-damaged buildings, producing a moderate class imbalance that I accounted for during training. Each image varied in resolution and scale, so the first step was to standardize the input. All images were resized to 128×128 pixels and normalized to the $[0, 1]$ range by dividing pixel values by 255. Normalization ensured stable gradient updates and faster convergence. Using TensorFlow's ImageDataGenerator, I automated this preprocessing and divided the data into 80 percent for training and 20 percent for validation, yielding approximately 17 thousand training and 4 thousand validation images. The 128×128 resolution struck a good balance between retaining sufficient spatial detail for pattern recognition and keeping computation feasible on the class virtual machine. Because the dataset already contained diverse examples—different lighting, camera angles, and degrees of structural damage—I avoided heavy augmentation such as flips or rotations. Visual inspection of random samples using Matplotlib confirmed that both classes were cleanly labeled and that the preprocessing pipeline functioned as intended. Damaged buildings typically showed collapsed roofs, standing water, or debris, whereas non-damaged structures appeared regular and intact. Establishing this consistent preprocessing was essential for fair comparison among models.

I experimented with three architectures of increasing complexity to study how network depth affected performance. The first model was a fully connected artificial neural network (ANN) that flattened each $128 \times 128 \times 3$ image into a 49,152-element vector and passed it through dense layers of 256 and 128 neurons with ReLU activation and a 0.3 dropout rate for regularization, followed by a single-node sigmoid output. Trained with the Adam optimizer (learning rate 1×10^{-4}) and binary cross-entropy loss, the ANN reached a best validation accuracy of 0.6646 (66 percent) at its first epoch before overfitting began. The learning curves showed that without convolutional layers the model failed to capture spatial relationships, memorizing superficial

textures instead. This baseline established that fully connected layers alone were insufficient for this type of image recognition.

To improve spatial feature extraction, I next implemented the classical LeNet-5 convolutional neural network, which introduces hierarchical receptive fields ideal for low-resolution imagery. My version comprised two convolution-pooling blocks—Conv2D (6, 5×5 , ReLU) followed by MaxPooling (2×2), then Conv2D (16, 5×5 , ReLU) followed by another MaxPooling (2×2)—and three dense layers of 120, 84, and 1 neurons respectively. This compact network, containing roughly 1.6 million trainable parameters, achieved a best validation accuracy of 0.7545 (75 percent) at epoch 2 while maintaining excellent generalization. The training and validation curves remained closely aligned, and no signs of severe overfitting appeared. LeNet-5 proved to be the optimal balance between representational power and computational cost, training quickly and using limited memory while learning meaningful spatial features such as roof integrity and shadow consistency.

Finally, I implemented a deeper alternate LeNet-5 inspired by Cao and Choe (2018), featuring convolutional layers with 32, 64, and 128 filters (3×3 kernels) interleaved with pooling layers and followed by a 512-unit ReLU dense layer and sigmoid output. This model had about 2.6 million parameters—larger than LeNet-5 but still tractable. Although it reached 0.7308 (73 percent) validation accuracy at epoch 3, it quickly began to overfit, achieving much higher training accuracy without further validation improvement. This confirmed that adding depth did not yield better performance given the dataset size and variability. Across all experiments the relative performances were consistent: ANN $\approx$ 66 percent, LeNet-5 $\approx$ 75 percent, and Alternate LeNet-5 $\approx$ 73 percent. The LeNet-5 model's simplicity, stability, and superior validation accuracy led me to select it as the final model for deployment. It provided strong generalization at minimal computational cost, making it suitable for production-style use on the constrained course environment.

After identifying LeNet-5 as the best performer, I exported it to `results/best_model.h5` and implemented the inference pipeline using FastAPI, a lightweight asynchronous web framework for serving machine-learning models. The application, contained in `src/app.py`, loads the saved model when the container starts and provides two HTTP endpoints. The GET `/summary` endpoint returns a JSON object describing the model name, input shape, and status flag indicating readiness. The POST `/inference` endpoint accepts either raw binary image bytes or multipart file uploads, converts the image to 128×128 RGB using Pillow, normalizes pixel values, performs inference, and returns a JSON response such as `{"prediction": "damage"}` or `{"prediction": "no_damage"}`. Comprehensive exception handling ensures clear 400 and 500 responses for invalid data or runtime errors, enhancing robustness and reproducibility.

To deploy the system, I containerized the server using Docker. The Dockerfile uses the python:3.11-slim base image and installs TensorFlow 2.15, FastAPI, Uvicorn, Pillow, NumPy, and Python-Multipart. The container exposes port 5000 and executes

uvicorn src.app:app --host 0.0.0.0 --port 5000 to launch the API. A docker-compose.yml file allows easy startup and shutdown, mapping the container's port 5000 to the host and ensuring reproducibility on any x86 environment. After building the image and running it with docker run -p 5000:5000 manasp123/project2_inference:latest, I verified the endpoints manually using curl commands. Both /summary and /inference responded correctly, and test images from the dataset produced accurate predictions for damaged and undamaged examples. These tests confirmed that preprocessing, model loading, and inference were functioning exactly as expected inside the container.

To validate official compliance, I executed the provided grader.py and start_grader.sh scripts on the UT Austin VM. The grader confirmed that the GET /summary and POST /inference endpoints matched the required format and successfully processed all images. Across multiple runs, the deployed model correctly classified between four and six of the six randomly selected test images, corresponding to an average accuracy of roughly 0.80 with several perfect 6/6 cases. These results verified that my inference server was fully functional and conformed to the course specification, with both endpoints passing automated validation. All grader logs reported proper JSON formatting and consistent server responses.

Overall, this project demonstrated the complete lifecycle of developing and deploying an intelligent system—from raw-data preprocessing and model prototyping through quantitative evaluation, selection, and real-time inference deployment. Among the three architectures tested, the LeNet-5 CNN achieved the best validation accuracy of approximately 75 percent, outperforming the ANN (66 percent) and alternate LeNet-5 (73 percent). The final Dockerized inference server consistently achieved around 80 percent accuracy on unseen test images and met all COE 379L grading requirements. The entire system, including the training notebook, model file, and source code, is hosted publicly at github.com/Manas2006/project2.

Through this project I gained practical experience with model evaluation, software packaging, and responsible AI deployment, reinforcing how theoretical machine-learning principles translate into robust real-world engineering systems.

Manas Pathak

University of Texas at Austin